

gRPC — requestObserver and responseObserver as Pipes

All 4 patterns with code and pipe ownership explained

The pipe concept

A StreamObserver is a pipe — an open channel through which multiple messages can flow. You do not create a new pipe for each message. You create ONE pipe and call `onNext()` on it as many times as you want.

More messages = more `onNext()` calls on the SAME pipe. Not more pipes.

Two pipes exist in gRPC — one in each direction:

requestObserver — pipe from client to server (carries `NumberRequest` messages)

responseObserver — pipe from server to client (carries `NumberResponse` messages)

The two rules that never change

Rule 1: `responseObserver` is ALWAYS 1. Always created by the framework. Always injected into the server method as a parameter. The server just uses it to write responses into.

Rule 2: `requestObserver` only EXISTS when the client streams. When it exists it is always 1 pipe. The server creates it and returns it. The client calls `onNext()` on it many times — one call per message.

Pattern 1 — Unary

Client sends 1 request. Server sends 1 response.

Observer	Count	Created by	Used by	How many messages
requestObserver	none	—	—	No pipe — request is a plain method argument
responseObserver	1	Framework	Server writes into it	Server calls onNext() once then onCompleted()

Client code

```
// NO requestObserver – just pass the request directly as an argument
NumberRequest req = NumberRequest.newBuilder().setNumber(5).build();

// blocking stub – returns response directly, no observer needed
NumberResponse resp = blockingStub.getNumber(req);
System.out.println(resp.getValue());
```

Server code (ServiceImpl)

```
public void getNumber(
    NumberRequest request,
    StreamObserver responseObserver) { // injected by framework

    // responseObserver = pipe TO client – use it to send the response
    responseObserver.onNext(
        NumberResponse.newBuilder().setValue(42).build());
    responseObserver.onCompleted(); // close the pipe
}
```

The client has no observer at all. It just calls the stub and gets a return value back. The server has one responseObserver pipe — calls onNext() once and closes it.

Pattern 2 — Server Streaming

Client sends 1 request. Server sends many responses.

Observer	Count	Created by	Used by	How many messages
requestObserver	none	—	—	No pipe — still a plain argument
responseObserver	1	Framework	Server calls onNext() many times	Same ONE pipe carries ALL response messages

Client code

```
// client creates observer to RECEIVE the stream of responses
StreamObserver myObserver = new StreamObserver<>() {
    public void onNext(NumberResponse r) {
        print(r.getValue()); // fires once per server onNext() call
    }
    public void onCompleted() { ... }
};

// ONE request argument + pass in your observer
asyncStub.getNumbers(req, myObserver);
```

Server code (ServiceImpl)

```
public void getNumbers(
    NumberRequest request,
    StreamObserver responseObserver) {

    // same ONE pipe – call onNext() many times
    for (int i = 1; i <= request.getCount(); i++) {
        responseObserver.onNext(
            NumberResponse.newBuilder().setValue(i).build());
    }
    responseObserver.onCompleted(); // done sending
}
```

The responseObserver pipe is still ONE pipe. It just carries more messages through it. Each call to onNext() sends one more message through the same open pipe.

Pattern 3 — Client Streaming

Client sends many requests. Server sends 1 response.

Observer	Count	Created by	Used by	How many messages
requestObserver	1	Server creates, returns it	Client calls onNext() many times	Same ONE pipe — client calls onNext() 3 times = 3 messages
responseObserver	1	Framework	Server calls onNext() ONCE inside onCompleted()	Server waits until client is done, then responds once

Client code

```
// client creates observer to receive the ONE final response
StreamObserver responseObserver = new StreamObserver<>() {
    public void onNext(NumberResponse r) {
        print("Final sum: " + r.getValue()); // fires ONCE
    }
};

// stub returns requestObserver – the server's pipe
StreamObserver requestObserver =
    asyncStub.sendNumbers(responseObserver);

// send many messages through ONE pipe – same requestObserver every time
requestObserver.onNext(NumberRequest.newBuilder().setNumber(10).build());
requestObserver.onNext(NumberRequest.newBuilder().setNumber(20).build());
requestObserver.onNext(NumberRequest.newBuilder().setNumber(30).build());
requestObserver.onCompleted(); // tell server we are done sending
```

Server code (ServiceImpl)

```
public StreamObserver sendNumbers(
    StreamObserver responseObserver) {

    // server creates and returns the request pipe
    return new StreamObserver() {
        int sum = 0;

        public void onNext(NumberRequest req) {
            // fires for EACH client message – 3 times in this example
            sum += req.getNumber();
        }
    };
}
```

```
public void onCompleted() {  
    // client pipe closed – NOW send the one response back  
    responseObserver.onNext(  
        NumberResponse.newBuilder().setValue(sum).build());  
    responseObserver.onCompleted();  
}  
};  
}
```

requestObserver is still ONE pipe even though 3 messages flow through it. The server does not respond inside `onNext()` — it accumulates. The response only happens inside `onCompleted()` when all messages have arrived.

Pattern 4 — Bidi Streaming

Client sends many. Server sends many. Both pipes open simultaneously.

Observer	Count	Created by	Used by	How many messages
requestObserver	1	Server creates, returns it	Client calls onNext() many times	ONE pipe — client sends many messages through it
responseObserver	1	Framework	Server calls onNext() many times inside its own onNext()	ONE pipe — server responds to each message immediately

Client code

```
// client creates observer to receive MANY responses
StreamObserver responseObserver = new StreamObserver<>() {
    public void onNext(NumberResponse r) {
        print("Sum: " + r.getValue()); // fires once per server response
    }
    public void onCompleted() { latch.countDown(); }
};

// stub returns requestObserver — ONE pipe to send on
StreamObserver requestObserver =
    asyncStub.getRunningSum(responseObserver);

// many messages, ONE pipe — same pattern as client streaming
requestObserver.onNext(NumberRequest.newBuilder().setNumber(10).build());
requestObserver.onNext(NumberRequest.newBuilder().setNumber(20).build());
requestObserver.onNext(NumberRequest.newBuilder().setNumber(30).build());
requestObserver.onCompleted();
latch.await(); // wait for server to finish
```

Server code (ServiceImpl)

```
public StreamObserver getRunningSum(
    StreamObserver responseObserver) {

    return new StreamObserver() {
        int sum = 0;

        public void onNext(NumberRequest req) {
            sum += req.getNumber();
            // KEY DIFFERENCE from Client Streaming:
```

```

// respond IMMEDIATELY inside onNext() not in onCompleted()
responseObserver.onNext(
    NumberResponse.newBuilder().setValue(sum).build());
}

public void onCompleted() {
    // just close the response pipe
    responseObserver.onCompleted();
}
};
}

```

Both pipes are open at the same time. Client sends through requestObserver, server immediately responds through responseObserver. The only difference from Client Streaming is WHERE the server calls responseObserver.onNext() — inside onNext() here, inside onCompleted() in Client Streaming.

Summary — all 4 patterns

Pattern	requestObserver	responseObserver	Key difference
Unary	Does not exist	1 — framework	Plain arg in, plain return out
Server Streaming	Does not exist	1 — framework	Server calls onNext() many times
Client Streaming	1 — server creates	1 — framework	Server responds in onCompleted()
Bidi Streaming	1 — server creates	1 — framework	Server responds in onNext() immediately

The count of observers never changes — always 1 requestObserver and 1 responseObserver per call. More messages means more onNext() calls on the same pipe, never more pipes.